

TECHNICAL SPECIFICATION

Creative Performance Dashboard — Implementation Spec

Everything needed to rebuild the owned-Meta-account creative performance dashboard for a different client, in a different environment, from scratch.

intel — owned Meta creative performance · 2026-08-11

Contents

1. What this thing is

2. Prerequisites

- 2.1 Meta access
- 2.2 Runtime

3. Required API endpoints

- 3.1 Ad-level insights (the metric spine)
- 3.2 Ad + creative metadata
- 3.3 Ad set targeting (audience facets)
- 3.4 Creative preview (best-fidelity asset)
- 3.5 Time-bucketed insights (series)
- 3.6 Asset download
- 3.7 API gotchas that cost real debugging time
- 4.1 owned_ads — one row per owned ad
- 4.2 ad_performance — metrics per ad per reporting window
- 4.3 ad_performance_series — weekly buckets (sparklines)
- 4.4 ad_daily — daily grain (timeline + early-read)
- 4.5 Shared tables the dashboard reads
- 4.6 Asset layout on disk

5. Ingest pipeline

- 5.1 Order of operations (per account, per window)
- 5.2 Normalization rules (these are the correctness core)
- 5.3 Attribution blob
- 5.4 Creative classification
- 5.5 Audience classification

6. Vision analysis layer

- 6.1 Schema
- 6.2 Confidence gate
- 6.3 Which creative gets analyzed
- 6.4 Running it with an API key
- 6.5 Running it without an API key

7. Dashboard build

- 7.1 Entry point
- 7.2 Window selection — the double-count guard
- 7.3 Prior-period comparison
- 7.4 Rankings window

7.5 Series projection onto a canonical timeline

7.6 Daily series is SPARSE

7.7 Shipped payload

8. Metric definitions (canonical)

8.1 Funnel

8.2 Scale or kill

8.3 Meta rating vs ROAS

8.4 Early read vs final scale

8.5 Creative tag manager

9. CLI + operational runbook

9.1 Commands

9.2 Multi-account refresh script

9.3 Ingesting a second, shorter window for rankings

9.4 Static web deploy

10. Porting checklist for a new client

11. Known limits — state these in the deliverable

Reference implementation in this repo:

Layer	File
Graph API client	src/intel/adapters/meta_account.py
Ingest orchestration	src/intel/adapters/meta_account_ingest.py
Schema + upserts	src/intel/storage.py
Dashboard build + client app	src/intel/synthesis/performance_dashboard.py
CLI surface	src/intel/cli.py:1315-1520
Vision taxonomy	src/intel/analysis/creative.py
No-API-key vision path	scripts/cc_vision_prep.py , scripts/cc_vision_write.py
Refresh runbook	scripts/refresh_jdsports_perf.sh
Static web deploy	scripts/build_site.py , NETLIFY.md

Live scale reference (JD Sports deployment, 4 accounts, 90-day window): 1,261 owned ads · 2,645 `ad_performance` rows · 5,691 weekly series rows · 18,074 daily rows · 1,047 creative assets · 419 vision-analyzed. Output is a single ~4 MB self-contained `index.html`.

1. What this thing is

A **static, single-file HTML dashboard** that joins first-party Meta ad-account performance (spend, impressions, clicks, conversions, revenue, ROAS) onto **vision-derived creative attributes** of the same ads, so you can answer *which creative treatments actually correlate with performance* — not just which ads exist.

Architecture in one line:

Meta Graph Marketing API → SQLite → vision analysis → one HTML file (all data inlined)

Three design decisions that drive everything else:

- All aggregation happens in the browser.** The build ships one record per ad (~1,300 rows) plus a filter/spec registry into a `<script>` tag; every table, chart, index and baseline is computed client-side. Reason: the filter bar has 5+ facets (brand × account × funnel stage × gender × geo × age × creative class), and pre-rendering a table per combination is combinatorial. It also keeps a *single* definition of each aggregation instead of one in Python and one in JS.
- Coverage is partial by construction and must be stated.** Catalog/DPA ads have no fixed creative to analyze and are typically the *majority of spend* (542 of 1,261 ads on the JD set). Every vision view is explicitly scoped to the analyzable subset and prints its spend coverage.
- Ad Library IDs and ad-account IDs are different namespaces.** `ad_archive_id` (public Ad Library) does **not** map to `ad.id` (owned account). There is no bridging API. Owned ads are re-ingested from the account side where

`ad.id` is the native key. The only weak bridge is `creative.effective_object_story_id`, whose prefix is the page id.

2. Prerequisites

2.1 Meta access

Requirement	Detail
Ad account access	The token's user must have at least Analyst role on each ad account (client Business Manager grants this via Business Settings → Partners, or a system user).
Permission scope	<code>ads_read</code> is sufficient. <code>ads_management</code> is not required.
App	A Meta app in Business/Advanced Access for <code>ads_read</code> . Dev-mode apps only read accounts owned by app admins/testers.
Token type	Long-lived user token or (better for automation) a system-user token from the client's Business Manager — it does not expire.
Graph version	Pinned to <code>v21.0</code> in <code>GRAPH</code> . Bump deliberately; field availability changes between versions.
Account ids	Numeric, without the <code>act_</code> prefix in config; the code prepends <code>act_</code> .

Env vars (see [.env.example](#)):

```
META_AD_LIBRARY_ACCESS_TOKEN= # required – the owned-account token (name is legacy)
ANTHROPIC_API_KEY=           # optional – vision analysis; see §6 for the no-key path
INTEL_DB_PATH=./data/<client>.db
INTEL_DATA_DIR=./data/<client>_assets
INTEL_COMPETITORS_FILE=./config/competitors_<client>.yaml
INTEL_VISION_MODEL=claude-sonnet-4-6
```

2.2 Runtime

- Python 3.11+ (3.13 used here)
- `httpx`, `click`, `rich`, `pyyaml`, `python-dotenv`, `Pillow`, `imagehash`
- `playwright` + `playwright install chromium` — **only** for ad-preview rendering. Skippable with `--no-previews` at the cost of the best creative asset.
- No web server. Output is a file. Optional static host for sharing (§9).

3. Required API endpoints

All against `https://graph.facebook.com/v21.0` . Every call carries `access_token` . All five are `GET` .

3.1 Ad-level insights (the metric spine)

GET /act_{account_id}/insights

Param	Value	Notes
level	ad	
fields	see <code>INSIGHT_FIELDS</code> below	
time_range	<code>{"since":"YYYY-MM-DD","until":"YYYY-MM-DD"}</code> JSON	preferred over <code>date_preset</code>
date_preset	last_90d	fallback when no explicit range
action_attribution_windows	<code>["1d_view","1d_click","7d_click","28d_click"]</code> JSON	
limit	100 with attribution windows, 500 without	load-bearing — see gotchas

INSIGHT_FIELDS :

ad_id, ad_name, campaign_id, campaign_name, adset_id, adset_name, impressions, spend, clicks, ctr, cpc, cpm, reach, frequency, actions, action_values, purchase_roas, quality_ranking, engagement_rate_ranking, conversion_rate_ranking, video_play_actions, video_thruplay_watched_actions, video_p25_watched_actions, video_p50_watched_actions, video_p75_watched_actions, video_p100_watched_actions

Pagination: follow `paging.next` verbatim — the cursor URL already carries every param, so **send no params on subsequent pages**.

Ads with no delivery in the window do not appear. Row count is "ads that ran", not "ads that exist".

3.2 Ad + creative metadata

GET /?ids={comma-separated ad ids}&fields={CREATIVE_FIELDS}

Batched by `ids` , **chunked at 50** (Graph's hard cap → `ID_CHUNK = 50`).

CREATIVE_FIELDS :

```
id,name,status,created_time,
creative{id,name,object_type,image_url,image_hash,video_id,thumbnail_url,
  title,body,link_url,call_to_action_type,object_story_id,
  effective_object_story_id,product_set_id,asset_feed_spec,object_story_spec}
```

A failed chunk is logged and skipped, never fatal — one bad chunk must not lose the rest of the account.

3.3 Ad set targeting (audience facets)

```
GET /?ids={comma-separated adset ids}&fields={ADSET_FIELDS}
```

ADSET_FIELDS :

```
id,name,optimization_goal,
targeting{age_min,age_max,genders,geo_locations,custom_audiences,
  excluded_custom_audiences,flexible_spec}
```

Fetch only the ad sets referenced by ingested ads. These accounts hold thousands of ad sets (2,225 in one) while only a few hundred delivered in a window; listing the whole `/act_x/adsets` edge with `targeting` expanded trips a Graph 500.

3.4 Creative preview (best-fidelity asset)

```
GET /{creative_id}/previews?ad_format=MOBILE_FEED_STANDARD
```

Returns `data[0].body` = an HTML `<iframe>` snippet. Regex the `src`, HTML-unescape it, then screenshot that URL with Playwright. DPA creatives legitimately return "Story Unavailable" — treat as expected, not an error, and skip the browser navigation entirely for `creative_class == 'dpa'`.

3.5 Time-bucketed insights (series)

```
GET /act_{account_id}/insights
  ?level=ad
  &fields=ad_id,impressions,spend,clicks,actions,action_values,video_play_actions
  &time_range={"since":..., "until":...}
  &time_increment={1|7}
  &limit=500
```

- `time_increment=7` → weekly buckets → `ad_performance_series` (sparklines, ~13 points/90 days)
- `time_increment=1` → daily → `ad_daily` (scale/kill timeline, early-read analysis)

Deliberately a **narrower field set** than §3.1: a bucketed row exists once per ad per period, so at daily grain the row count is $\sim 90\times$ the ad count and every stored column is paid for 90 times.

3.6 Asset download

Plain HTTP GET on the signed CDN URLs from `image_url` / `thumbnail_url`. The sizing is baked into the signed URL — **stripping the `stp=crop` param to get a larger image returns 403**. Video `source` bytes are permission-gated, so a $\sim 160\times 160$ `thumbnail_url` is the ceiling for video assets without a rendered preview.

3.7 API gotchas that cost real debugging time

Symptom	Cause / fix
<code>400</code> + "Service temporarily unavailable" subcode <code>1504044</code> , marked <code>is_transient: false</code>	The flag is wrong . Identical request succeeds on retry. Treat as retryable.
"please reduce the amount of data" on insights	Per-window attribution multiplies conversion payload per row. Drop <code>limit</code> to 100.
Daily (<code>time_increment=1</code>) 90-day pulls reliably 500	Load failure, not transient — retries don't help. Chunk the window (15 days default). Write each chunk as it arrives so a mid-run failure keeps what's already fetched.
<code>403</code> "request limit reached" across accounts	App-level rate limit. Put a 20s cooldown between accounts in batch scripts.
<code>quality_ranking</code> is <code>UNKNOWN</code> for everything	Meta only defines rankings over <i>recent</i> delivery. Measured: 25/25 UNKNOWN at 90 days; 144/253 ranked at 30 days. Ingest a shorter window too, and scope the ranking view to it.
<code>video_continuous_2_sec_watched_actions</code> is always 0	Not populated on these accounts. Use the <code>video_view</code> action (3-second) instead.
<code>video_play_actions</code> $\approx 92\%$ of impressions	Autoplay initiations, not engagement. Never use as a hook-rate numerator; use it as the <i>flag</i> that the ad served video.
Same window ingests under different <code>date_start</code> per account	Accounts sit in different timezones; Meta echoes shifted dates. Overwrite <code>date_start</code> / <code>date_stop</code> with the requested range before storing, or each account lands under a different key and re-ingest won't overwrite.
<code>dda</code> (data-driven) attribution returns identical values to default	These accounts are already on DDA. Dropped — it added no information and doubled rate-limit pressure. True incrementality needs a conversion-lift holdout, not an attribution window.
7d/28d view -through windows come back empty	Retired by Meta post-iOS-14. Only <code>1d_view</code> survives. Requesting them just adds empty keys.
## 4. Data model	

SQLite. All DDL is `CREATE TABLE IF NOT EXISTS` + `ALTER TABLE` forward-migrations run idempotently on every `connect()` — see `_migrate_owned_perf_tables`.

4.1 owned_ads — one row per owned ad

```
CREATE TABLE IF NOT EXISTS owned_ads (
  platform_ad_id TEXT PRIMARY KEY,      -- the ad account's native ad.id
  competitor_id TEXT REFERENCES competitors(id),
  account_id TEXT,
  account_name TEXT,
  ad_db_id INTEGER REFERENCES ads(id),  -- link into the shared ads table
  ad_name TEXT,
  campaign_id TEXT,  campaign_name TEXT,
  adset_id TEXT,    adset_name TEXT,
  creative_id TEXT,
  object_type TEXT,
  creative_class TEXT,                  -- analyzable | dpa | no_asset
  title TEXT, body TEXT, cta_type TEXT, link_url TEXT,
  product_set_id TEXT,
  effective_object_story_id TEXT,       -- '<page_id><post_id>' - the only brand bridge
  created_time TEXT,                   -- ad creation = "launch date"
  audience_stage TEXT,                -- retargeting | lookalike | interest | prospecting_broad |
  custom_audience_other | unknown
  audience_gender TEXT,               -- all | men | women | mixed
  audience_age TEXT,                 -- e.g. '18-65' | 'unspecified'
  audience_geo TEXT,                 -- national | dma | local_radius | regional | unspecified
  audience_name TEXT,                -- raw adset name
  audience_custom TEXT,              -- ';' -joined custom-audience names (first 4)
  optimization_goal TEXT,
  raw_json TEXT,
  ingested_at TEXT
);
CREATE INDEX idx_owned_ads_comp      ON owned_ads(competitor_id);
CREATE INDEX idx_owned_ads_stage     ON owned_ads(audience_stage);
CREATE INDEX idx_owned_ads_class     ON owned_ads(creative_class);
CREATE INDEX idx_owned_ads_account   ON owned_ads(account_id);
```

Audience facets are **denormalised onto the ad** (an ad belongs to exactly one ad set, and the dashboard filters ads directly).

Upsert is idempotent on `platform_ad_id`; audience columns use `COALESCE(excluded.x, owned_ads.x)` so a re-ingest that skips ad-set fetch does not blank them.

4.2 ad_performance — metrics per ad per reporting window

```
CREATE TABLE IF NOT EXISTS ad_performance (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

platform_ad_id TEXT NOT NULL,
competitor_id TEXT, account_id TEXT,
date_start TEXT, date_stop TEXT,
impressions REAL, spend REAL, clicks REAL,
ctr REAL, cpc REAL, cpm REAL, reach REAL, frequency REAL,
link_clicks REAL, purchases REAL, revenue REAL, roas REAL,
thruplays REAL, video_p25 REAL, video_p50 REAL, video_p75 REAL, video_p100 REAL,
video_3s REAL,           -- 3s views: scroll-stop numerator
video_plays REAL,         -- autoplay initiations: "served video at all" flag
landing_page_views REAL, view_content REAL,
add_to_cart REAL, initiate_checkout REAL,
quality_ranking TEXT,     -- ordinal buckets, NOT numbers
engagement_rate_ranking TEXT,
conversion_rate_ranking TEXT,
attribution_json TEXT,    -- {window: {pu,rv,atc,ic,lpv,vc}}
extra_json TEXT,         -- raw actions/action_values blobs
fetched_at TEXT,
UNIQUE(platform_ad_id, date_start, date_stop)
);
CREATE INDEX idx_ad_perf_comp ON ad_performance(competitor_id);
CREATE INDEX idx_ad_perf_ad   ON ad_performance(platform_ad_id);

```

The `UNIQUE` constraint is what makes ingest **safely repeatable** — re-running a window overwrites instead of double-counting spend.

`attribution_json` is a blob, not columns: 6 metrics $\times$ 6 windows would be 36 columns, and the dashboard ships it to the browser and re-weights client-side.

4.3 `ad_performance_series` — weekly buckets (sparklines)

```

CREATE TABLE IF NOT EXISTS ad_performance_series (
  platform_ad_id TEXT NOT NULL,
  competitor_id TEXT, account_id TEXT,
  bucket_start TEXT NOT NULL,
  impressions REAL, spend REAL, clicks REAL,
  purchases REAL, revenue REAL, video_3s REAL, video_plays REAL,
  fetched_at TEXT,
  PRIMARY KEY (platform_ad_id, bucket_start)
);

```

4.4 `ad_daily` — daily grain (timeline + early-read)

```
CREATE TABLE IF NOT EXISTS ad_daily (
  platform_ad_id TEXT NOT NULL,
  competitor_id TEXT, account_id TEXT,
  day TEXT NOT NULL,
  impressions REAL, spend REAL, clicks REAL,
  purchases REAL, revenue REAL, video_3s REAL, video_plays REAL,
  fetched_at TEXT,
  PRIMARY KEY (platform_ad_id, day)
);
```

Do not merge 4.3 and 4.4. `ad_performance_series` is keyed `(platform_ad_id, bucket_start)` with **no bucket-width column**. A daily row and a weekly row sharing a start date collide on the primary key — the daily value silently overwrites the weekly one and the sparklines plot a single day against a 13-week axis. Separate tables is the fix.

4.5 Shared tables the dashboard reads

Owned ads are also written into the project's generic `ads` / `creatives` tables under `source = 'meta_owned'`, so the existing vision pipeline and phash dedup work unchanged. If you are building standalone, you need these two:

```
CREATE TABLE ads (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  competitor_id TEXT NOT NULL,
  source TEXT NOT NULL DEFAULT 'meta',    -- 'meta_owned' for this lane
  ad_archive_id TEXT NOT NULL UNIQUE,    -- holds platform_ad_id for owned ads
  page_id TEXT, page_name TEXT,
  first_seen TEXT NOT NULL, last_seen TEXT NOT NULL,
  active INTEGER NOT NULL DEFAULT 1,
  start_date TEXT, end_date TEXT,
  body_text TEXT, cta_type TEXT, link_url TEXT,
  publisher_platforms TEXT, raw_json TEXT, serp_position_rank INTEGER
);

CREATE TABLE creatives (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  ad_id INTEGER REFERENCES ads(id),
  competitor_id TEXT REFERENCES competitors(id),
  source TEXT NOT NULL DEFAULT 'meta',
  asset_type TEXT NOT NULL,    -- ad_preview | image | video_thumb
  asset_path TEXT NOT NULL,    -- local path under DATA_DIR
  phash TEXT,                  -- perceptual hash for visual dedup
```

```

    analysis_json TEXT,           -- vision-model output
    analyzed_at TEXT
);
CREATE INDEX idx_creatives_ad    ON creatives(ad_id);
CREATE INDEX idx_creatives_comp ON creatives(competitor_id);

```

Plus a `competitors(id, name, vertical, priority, meta_json)` lookup for brand labels.

4.6 Asset layout on disk

```

{INTEL_DATA_DIR}/creative_owned/{competitor_id}/{platform_ad_id}/
  preview.png      # rendered ad preview – best fidelity
  image_0.jpg      # direct image_url
  video_thumb_1.jpg # thumbnail_url (a real video frame, ~160x160)

```

5. Ingest pipeline

5.1 Order of operations (per account, per window)

1. `fetch_insights(account, since, until, attribution_windows=ATTR_WINDOWS)`
2. `normalize_insight()` per row → flat float columns + attribution blob
3. pin `date_start/date_stop` to the REQUESTED range (timezone fix)
4. `fetch_ad_meta(ad_ids)` → creative nodes, chunks of 50
5. `fetch_adsets(adset_ids referenced)` → targeting → `classify_audience()`
6. sort by spend DESC → capped preview runs cover top spenders
7. per ad:
 - `classify_creative()` → analyzable | dpa | no_asset
 - `creative_copy()` → title/body/cta/link, preferring `asset_feed_spec` variants
 - `upsert_ad()` → shared ads table, `source='meta_owned'`
 - `upsert_owned_ad()`
 - `upsert_ad_performance()`
 - `extract_assets()` → `download_asset()` → `upsert_creative()`
8. batch: `preview_iframe_url()` per analyzable creative
 - `render_previews()` in ONE browser session
 - `upsert_creative(asset_type='ad_preview')`
9. `spend_coverage()` → report analyzable % honestly

Series ingest is a separate pass (`perf-series`), run after the summary ingest.

5.2 Normalization rules (these are the correctness core)

Conversions live in `actions` / `action_values` as lists of `{action_type, value, 1d_view, 7d_click, ...}`. Everything arrives as strings.

`_action_value(actions, names)` takes the **FIRST match in the given name order, not the sum**. Meta reports the same conversion under several attribution surfaces; summing the aliases counts one cart-add up to four times.

Alias precedence — **omni_* first, deliberately**:

Metric	Action types, in order
purchases / revenue	<code>omni_purchase</code> , <code>purchase</code> , <code>offsite_conversion.fb_pixel_purchase</code>
landing_page_views	<code>omni_landing_page_view</code> , <code>landing_page_view</code>
view_content	<code>omni_view_content</code> , <code>view_content</code>
add_to_cart	<code>omni_add_to_cart</code> , <code>add_to_cart</code>
initiate_checkout	<code>omni_initiated_checkout</code> , <code>initiate_checkout</code>
link_clicks	<code>link_click</code>

Metric	Action types, in order
video_3s	<code>video_view</code> (from <code>actions</code>)
video_plays	<code>video_view</code> (from <code>video_play_actions</code>)
thruplays / p25...p100	<code>video_view</code> (from the respective <code>video*_watched_actions</code>)

`omni_purchase` wins 584/584 rows on the reference accounts. A funnel built on the web-only surface would end on a smaller number than the ROAS tiles and read as a bug. Measured: omni and web-only are identical for cart-add and checkout, and diverge only at purchase (256,339 vs 222,731) — that gap is in-store/offline conversions. **Footnote it; don't hide it by switching surfaces mid-funnel.**

`purchase_roas` arrives as a single-element list → take `[0].value`.

5.3 Attribution blob

`build_attribution()` produces:

```
{
  "default": {"pu":0,"rv":0,"atc":0,"ic":0,"lpv":0,"vc":0},
  "1d_view": {...}, "1d_click": {...}, "7d_click": {...}, "28d_click": {...}
}
```

`default` reads Meta's `value` key, so it is **byte-identical to the flat columns** — switching the dashboard to "Account default" reproduces a pre-attribution build exactly. A window key absent from an action means *zero credit in that window*, not missing data. Click windows are cumulative ($1d \subseteq 7d \subseteq 28d$).

Only conversions are attributed. Spend, impressions, clicks, CTR, CPM and scroll-stop never vary by window — the UI must say so.

5.4 Creative classification

```
def classify_creative(creative):
    afs = creative.get("asset_feed_spec") or {}
    if creative.get("video_id") or creative.get("image_url"): return "analyzable"
    if afs.get("videos") or afs.get("images"): return "analyzable"
    oss = creative.get("object_story_spec") or {}
    tpl = str((oss.get("template_data") or {}).get("name") or "")
    if creative.get("product_set_id") or "{{product." in tpl: return "dpa"
    return "no_asset"
```

Order matters: a dynamic-creative ad can carry both a `product_set_id` **and** real assets in `asset_feed_spec` — the assets win, because you can look at it.

DPA ads are still ingested and still carry performance; they can be reported on **metadata** attributes (copy, CTA, format, campaign, audience) but never on vision attributes.

5.5 Audience classification

Meta has **no "is this retargeting" flag**. It is inferred from the ad set's targeting spec first, falling back to name patterns (these accounts encode strategy in ad set names: "Advantage+ | PRO | DMAs", "90d_Site Visitors", "US - W - 13-65").

Funnel stage precedence: 1. custom audiences matching retarget hints (site_visitor , cart , purchaser , engaged , crm , dpa_ , ...) → **retargeting** 2. custom audiences matching lookalike hints (lookalike , la_ , (us, , %)) → **lookalike** 3. same two hint sets against the ad set **name** 4. interest hints or a flexible_spec present → **interest** 5. name contains advantage / broad / pro → **prospecting_broad** 6. any custom audience at all → **custom_audience_other** 7. age_min or geo_locations present → **prospecting_broad** 8. otherwise → **unknown**

Anything unclassifiable stays unknown rather than defaulting into prospecting, which would silently inflate it.

Lookalikes rank as prospecting, not retargeting — they're modelled off engagers but still point at strangers.

Gender: genders == [1] → men, [2] → women, empty/None → all, else mixed. Geo scope: geo_markets → dma ; custom_locations / places → local_radius ; regions / cities / zips → regional ; countries → national .

Re-tune the hint lists per client — they encode one agency's ad set naming conventions. This is the single most client-specific piece of logic in the system.

6. Vision analysis layer

6.1 Schema

The full taxonomy lives in `CREATIVE_TAXONOMY_PROMPT` ([src/intel/analysis/creative.py:16](#)). It returns strict JSON. The dashboard reads these fields:

Group	Keys
Scalar	<code>production_style</code> , <code>photography_style</code> , <code>product_emphasis</code> , <code>hook_style</code> , <code>emotional_vs_rational</code> , <code>aspect_ratio_guess</code> , <code>background_color</code> , <code>model_gender</code> , <code>logo_visible</code> , <code>before_after_present</code>
Nested (dot path)	<code>text_overlay.density</code> , <code>text_overlay.copy_lean</code> , <code>urgency_cues.present</code> , <code>casting.people_visible</code>
List (fan-out)	<code>value_props</code> , <code>key_features</code> , <code>products_visible</code> , <code>seasonal_tags</code>
Meta	<code>confidence</code> (0-1), <code>summary_one_line</code> , <code>notable_text</code> , <code>creative_context</code>

Booleans are normalised to "yes" / "no" strings before bucketing.

6.2 Confidence gate

```
MIN_ANALYSIS_CONFIDENCE = 0.45
```

Meta caps thumbnail downloads hard — some arrive at 64×64, where nothing beyond rough colour is legible and the analyser correctly self-reports ~0.3 confidence. Below-threshold analyses are **excluded from attribute rollups entirely**. Letting them vote manufactures attribute distributions out of unreadable pixels, which is worse than a smaller honest sample. Analyses with no `confidence` field (older runs) pass.

6.3 Which creative gets analyzed

One ad can have several creative rows (dynamic creative serves multiple variants; a rendered preview sits alongside the raw asset). Selection:

```
SELECT c.asset_type, c.analysis_json, c.asset_path
FROM creatives c JOIN owned_ads oa ON oa.ad_db_id = c.ad_id
WHERE oa.platform_ad_id = ? AND c.analysis_json IS NOT NULL
ORDER BY CASE c.asset_type WHEN 'ad_preview' THEN 0 ELSE 1 END, c.id
LIMIT 1
```

`ad_preview` wins because it is the ad **as actually served**. If nothing is analyzed, fall back to any asset at all so the drill-down isn't blank.

6.4 Running it with an API key

```
intel analyze-creatives --competitor <brand> --limit 200
```

6.5 Running it without an API key

The reusable subagent path (used across every deployment in this repo):

```
INTEL_DB_PATH=data/<client>.db python scripts/cc_vision_prep.py /tmp/tasks.json \
  --competitor <brand> --max-image 200 --max-video 40
# → dispatch each task to a vision-capable agent, collect {creative_id: analysis}
INTEL_DB_PATH=data/<client>.db python scripts/cc_vision_write.py results.json
```

`cc_vision_prep` adds the cost controls a keyed run gets for free: **phash dedup** (one representative per distinct image, analysis propagated to visually-identical siblings *within an asset-type family* — a video's first frame can phash-collide with a still, and their schemas differ), popularity ranking, and per-brand caps. `ad_preview` maps to `creative_context: meta_ad` — same schema as a scraped Meta ad; the surrounding Facebook chrome is the only difference.

7. Dashboard build

7.1 Entry point

```
build_performance_dashboard(conn, *, out_dir: Path,
                           competitor_ids: list[str] | None = None,
                           min_impressions: int = 1000) -> dict | None
```

Writes `{out_dir}/index.html`. Returns `{path, brands, ads, spend, analyzed}` or `None` if no data.

7.2 Window selection — the double-count guard

`ad_performance` can legitimately hold several overlapping windows (a 90-day pull *and* a 30-day one). **Summing across them double-counts every ad in both.**

```
def _pick_windows(conn):
    # reporting window = the WIDEST window ending at the LATEST date_stop
    # comparison window = the most recent window ending on or before it starts
```

The metric join is then scoped with `AND p.date_start = ? AND p.date_stop = ?`.

7.3 Prior-period comparison

The comparison is **not** "these same ads, earlier." Most current ads did not exist in the prior period, so that framing reports every new ad as infinite growth. `_fetch_prior()` sums over the ads that actually ran *then*, carrying the same filter facets (`b`, `ac`, `cl`, `stage`, `gen`, `age`, `geo`) plus their own `attribution_json`, so period-over-period deltas re-weight on the same window as the current side.

7.4 Rankings window

`_fetch_rankings()` does **not** read the main reporting window. It picks whichever ingested window has the most non-`UNKNOWN` `quality_ranking` rows, and reports ROAS from that same window so both axes of the rating plot describe the same period. The chosen window is shipped as `RANKWIN` and printed in the UI.

7.5 Series projection onto a canonical timeline

Meta only returns buckets in which an ad delivered, so each ad's raw series has a different length and start. **Element-wise summing would add week 3 of one ad to week 1 of another.** Every ad is projected onto the full sorted set of `bucket_start` values, zero-filled — which is also the truthful value.

7.6 Daily series is SPARSE

A dense matrix is $1,261 \text{ ads} \times 90 \text{ days} \times 8 \text{ metrics}$ of mostly zeros. Each ad instead carries only the days it ran:

```
D: [[dayIdx, spend, purchases, revenue, impressions, video_3s, video_plays, clicks], ...]
```

indexed against the global `DAYS` array. A day absent from the list is exactly zero delivery. The browser accumulates.

7.7 Shipped payload

Ten globals in one `<script>` tag:

Global	Contents
ADS	one record per ad (below)
PADS	prior-period records (facets + totals + <code>attr</code> only)
FILTERS	<code>[[field, label], ...]</code> — the filter bar facets
META_SPECS	metadata attributes known for every ad
TAIL_SPECS	campaign / ad set — rendered last (long tail, account structure not creative)
VISION_SPECS	<code>[key, label, 'scalar' 'list', isVision]</code>
TAG_META	tag descriptions + allowed values (mirrors the vision prompt)
TAG_SAVED	overrides loaded from <code>creative_tags.json</code> if present next to the output
BUCKETS, DAYS, RANKWIN	axes + the ranking window
MINIMP, MAXC	min-impressions floor, max cards per expanded bucket (60)

Ad record (short keys — the payload is inlined, so key length is ~15% of file size):

```
{
  id, nm,                // platform_ad_id, ad name
  b, ac, cp, an,         // brand, account, campaign, ad set
  cta, ot, cl,           // CTA type, object type, creative class
  stage, gen, age, geo, opt, // audience facets + optimisation goal
  at, img,               // asset type, asset path
  sp, im, ck, pu, rv,     // spend, impressions, clicks, purchases, revenue
  v3, vp, tp, lc, atc, ic, // 3s views, video plays, thurplays, link clicks, cart, checkout
  cr,                    // created_time[:10] – launch date
  attr: {window: {pu,rv,atc,ic,lpv,vc}}, // optional
  R: {q,e,c,sp,rv,pu,im}, // optional: Meta rankings + that window's metrics
  D: [[dayIdx, sp, pu, rv, im, v3, vp, ck], ...], // optional: sparse daily
  s: {im,sp,ck,pu,rv,v3,vp}, // optional: weekly series arrays
  A: {attrKey: value | [values]} // optional: vision attributes
}
```

Note: `landing_page_views` and `view_content` are stored in SQL but **not** shipped — the funnel uses link-click → cart → checkout → purchase.

Series ship **component metrics only**. Derived rates (CTR, ROAS, scroll-stop) are recomputed per bucket in the browser so a filtered sparkline stays exact rather than averaging pre-computed per-ad rates.

8. Metric definitions (canonical)

All aggregates are **impression-weighted, not row-averaged**. Averaging per-ad CTRs lets a 200-impression ad swing the mean as hard as a 2M-impression one.

```
CTR          = 100 * Σclicks / Σimpressions
CPM          = 1000 * Σspend / Σimpressions
CPC          = Σspend / Σclicks
ROAS         = Σrevenue / Σspend
CPA          = Σspend / Σpurchases
CVR          = 100 * Σpurchases / Σclicks    ← per CLICK, matching CTR's chain
Scroll-stop  = 100 * Σvideo_3s / Σimpressions_of_video_ads_only
Cost / 3s view = Σspend / Σvideo_3s
```

Scroll-stop's denominator is the load-bearing detail. Static and catalog-image ads can never register a 3-second view, so including their impressions drags the rate toward zero for reasons unrelated to the creative. Only ads with `video_plays > 0` contribute impressions, and the count of such ads (`vads`) is surfaced in brackets next to every value so a thin video sample is visible.

Indices: `ctr_index = 100 * bucket.ctr / baseline.ctr`, same for ROAS. `100` = the baseline **for the current filter**, so indices re-base as you narrow.

Bucket floor: any attribute value with fewer than `min_impressions` (default 1,000) impressions is dropped. A table with fewer than 2 surviving buckets is not rendered at all — there's nothing to compare against.

8.1 Funnel

```
STAGES = [Impressions, 3s hook*, Thruplay*, Link click, Add to cart, Checkout, Purchase]
          ^ video-denominator stages
```

Two modes: stage as % of **impressions** (its own denominator), or as % of the **previous step**. Step conversion is **suppressed (null, not a fake number)** when crossing the video/non-video denominator boundary.

Comparison axes are **brand + creative attributes only**. Campaigns, ad sets and audience targeting are deliberately excluded — the funnel answers "which creative treatment converts better", not "which line item spent more". Top 8 values per attribute (`SEG_CAP`).

8.2 Scale or kill

Every ad plotted as spend (x) vs efficiency (y), cumulative to a selected day, with a confidence band around a target.

Metric registry:

Metric	Lower is better	Video only	Formula
CPA	yes	no	sp/pu
ROAS	no	no	rv/sp
CPM	yes	no	$1000*sp/im$
Cost / 3s view	yes	yes	$sp/v3$

Zone band — a confidence band, not a fixed threshold. With $K = 2.0$ and $n = spend / target$ (the number of events expected at target):

```
f      = 1 + K / sqrt(max(n, 0.25))
kill   = target * f      scale = target / f      (cost metrics)
kill   = target / f      scale = target * f      (ROAS – direction flips)
```

The band tightens as an ad accumulates spend: a \$50 ad needs to be wildly off to be called; a \$50,000 ad is judged tightly.

No-conversion rule. An ad with zero purchases has no CPA, but that is not "no information". Once it has spent enough to have produced `NOCONV_N = 3` conversions at target and produced none, that **is** the finding → classify as KILL. Without this, the worst ads on the account sit in WAIT forever.

Target defaults to the account average for the active metric, with a "–5/10/15/20% vs avg" tightener. The note always prints the effective absolute target so direction is never ambiguous.

Launch cohort filter uses `created_time`, not first delivery. 166 of the reference ads were built before the window opened; filtering on first-delivery would label every one a fresh launch.

Timeline animates cumulative spend/efficiency day by day. During playback, per-zone sections (3 charts + a table each) are skipped and refreshed only on pause — rebuilding them $\sim 9\times$ /second was the stutter.

8.3 Meta rating vs ROAS

Ordinal buckets (`ABOVE_AVERAGE` , `AVERAGE` , `BELOW_AVERAGE_10` , ...) on one axis, ROAS on the other, scoped to `RANKWIN` (§7.4). Toggle across quality / engagement / conversion ranking.

8.4 Early read vs final scale

"Does an ad's first N days predict how far it scales?"

- **Early window:** each ad's first N days from its own first delivering day (slider, 1–30).
- **Signal (x):** CPC (default), CPM, CTR, CVR or ROAS over that early window.
- **Outcome (y):** final spend, or post-window spend (`final – early` , floored at 0).
- **Statistic:** Spearman ρ (average-rank, ties averaged → Pearson on ranks). Requires $n \geq 4$.
- **Gate:** ads below `ER_MINCLK` early clicks (default 10) are excluded — a 3-click CTR is noise.

- **Views:** scatter · sensitivity (ρ as the window widens 1→30 days, both outcomes) · quintiles (median outcome per early-signal quintile).

The **final spend** outcome shares the early dollars with the signal, which inflates ρ mechanically; the **post-window** outcome strips them out. Offering both is the honest presentation.

8.5 Creative tag manager

A form editor over the taxonomy. Renaming a value **merges** its bucket; hiding one removes it from every view; adding a value or tag is a **definition for the next analysis run** (so it shows zero ads). Edits are localStorage-scoped and apply across tables, charts, funnel axes and nav at once. They never re-tag any ad.

Export produces:

```
{"version": 1,
  "tags": [{"key","label","kind":"single|multi","description",
            "enabled","custom","values":[],"removed_values":[],"ads_tagged":0}],
  "overrides": { ...raw editor state... }}
```

`tags` is the resolved taxonomy — hand it to the vision prompt. `overrides` is re-importable. **Save the file as `creative_tags.json` next to the dashboard and the next build picks it up as everyone's default** (`build_performance_dashboard` reads `out_dir / "creative_tags.json"` and ships it as `TAG_SAVED`).

Audience and account-structure attributes are not editable here — they come from Meta's API, not the creative analysis.

9. CLI + operational runbook

9.1 Commands

```
# 1. summary ingest – one call per account
intel perf-ingest --account 263673744705629 --competitor jdsports \
  --account-name "JD Sports US Brand & COOP" \
  --since 2026-04-30 --until 2026-07-29 --max-previews 30
# --no-previews      skip browser rendering entirely (much faster)
# --max-previews N    cap renders; highest-spend ads render first
# --days 90          instead of --since/--until

# 2. weekly series (sparklines)
intel perf-series --account <id> --competitor <brand> \
  --since 2026-04-30 --until 2026-07-29 --increment 7

# 3. daily series (scale/kill timeline + early read) – chunked automatically
intel perf-series --account <id> --competitor <brand> \
  --since 2026-04-30 --until 2026-07-29 --increment 1
# --chunk-days N      default 15 for daily, whole window for weekly

# 4. vision analysis (§6)

# 5. build
intel perf-dashboard --competitor jdsports --competitor finishline \
  --out reports/<client>/<date>/performance-dashboard \
  --min-impressions 1000 --open
```

9.2 Multi-account refresh script

Model on [scripts/refresh_jdsports_perf.sh](#). Shape:

```
export INTEL_DB_PATH="$ROOT/data/<client>.db"
export INTEL_DATA_DIR="$ROOT/data/<client>_assets"
ACCOUNTS=( "acct_id|competitor_id|Human Name|--max-previews 30" ... )
# three passes in order: summary → weekly → daily
# 20s cooldown between every account call – the app-level rate limiter is real
```

Run the passes in that order and never in parallel across accounts.

9.3 Ingesting a second, shorter window for rankings

Because `quality_ranking` is UNKNOWN at 90 days, run a **second summary ingest at 30 days** into the same DB. `_pick_windows` still selects the 90-day window as the reporting period (widest ending at the latest stop), and `_fetch_rankings` independently picks the 30-day one because it has ranked rows.

9.4 Static web deploy

The perf dashboard is **self-contained** (`build_site.py` lists it as having no asset refs to rewrite) *when previews are disabled* — but with previews it embeds local image paths. `scripts/build_site.py` copies referenced assets into `dist/<deployment>/<date>/assets/` and rewrites every reference to `../assets/...`.

```
python3 scripts/build_site.py      # latest date per deployment
./scripts/deploy_netlify.sh        # deploy LOCALLY, not via git CI
```

Deploy locally, not from git CI — `data/*_assets/` is gitignored, so a CI build from a checkout 404s the images. See [NETLIFY.md](#).

10. Porting checklist for a new client

- ☐ **Get the token.** Analyst role on every account + `ads_read` . Prefer a Business-Manager system-user token (non-expiring).
- ☐ **List the ad account ids** (numeric, no `act_`) and map each to a brand id. One brand can own several accounts (JD Sports has 2, Finish Line has 2).
- ☐ **New isolated deployment.** Set `INTEL_DB_PATH=data/<client>.db` and `INTEL_DATA_DIR=data/<client>_assets` . Never share a DB across clients — the window-selection logic scans the whole `ad_performance` table.
- ☐ **Register brands** in `config/competitors_<client>.yaml` with `id / name / vertical` , so `competitor_id` FKs resolve. Point at it with `INTEL_COMPETITORS_FILE=config/competitors_<client>.yaml` , then `intel init` .
- ☐ **Pick the window.** 90 days is the default; it must be long enough to contain a meaningful prior period if you want deltas. Ingest the prior period as its own window (same length, ending the day before the current starts).
- ☐ **Retune** `classify_audience` **hint lists** to this agency's ad set naming conventions. Verify by spot-checking the `audience_stage` distribution against what the account team believes they're running. This is the #1 source of wrong-looking numbers on a new account.
- ☐ **Check the conversion event names.** If the client's pixel uses custom conversions rather than standard `purchase` , extend `_ATTR_METRICS` and the alias tuples in `normalize_insight` . Verify total purchases against Ads Manager before showing anyone.
- ☐ **Check whether revenue exists at all.** Lead-gen and app accounts have no `action_values` — ROAS/revenue/CPA tiles are meaningless. Swap the scale/kill default metric to CPM or cost-per-lead and remove the revenue tiles.
- ☐ **Run coverage first.** `perf-ingest` prints the analyzable-spend %. Under ~40%, lead with the metadata tables (CTA, format, campaign, audience) and frame the vision tables as a subset read.
- ☐ **Retune the vision taxonomy** for the vertical (`products_visible` and `seasonal_tags` are open-vocabulary; `key_features` is retail-flavoured). Use the tag manager to draft it, export, fold into the prompt, re-run.
- ☐ **Set** `min_impressions` to something sane for the account's scale. 1,000 suits a multi-million-impression account; drop it for smaller ones or every table vanishes.
- ☐ **Verify against Ads Manager** on total spend, impressions and purchases for the exact window before it goes to the client. Timezone and attribution are the usual sources of a mismatch.

11. Known limits — state these in the deliverable

1. **Correlation, not causation.** Attributes co-vary with budget, bidding, placement and audience. The dashboard says so in its footer; keep that.
2. **Partial creative coverage.** DPA is usually the majority of spend and has no analyzable creative. Every vision view covers the analyzable subset only.
3. **Meta rankings are ordinal and short-window.** They cannot be averaged, and they don't exist for a 90-day pull.
4. **Attribution windows are not incrementality.** They re-slice the same conversions. Lift requires a holdout study.
5. **Scale/kill zones are a confidence band around a target**, not a decision rule. The band is a heuristic ($K = 2.0$), tuned by eye.
6. **Early-read ρ on "final spend" is mechanically inflated** because the early dollars are inside the outcome. Read the post-window series alongside it.
7. **Video asset fidelity is capped** at $\sim 160 \times 160$ without rendered previews, which is why some analyses self-report low confidence and get dropped.
8. **Funnel purchase counts include offline/in-store** (omni surface) while cart/checkout are web-equivalent. The gap is real, not a bug.
9. **The dashboard is a snapshot.** Re-running ingest for the same window overwrites in place; there is no history of what a metric read last week.